

МФТИ

Алгоритмы и структуры данных II

Семинар 02

Динамическое программирование (2)

Весна 2026

Строгие подробные решения с доказательствами

Содержание

Обозначения и соглашения	2
1 Задача 1. Операции с битовыми масками	3
2 Задача 2. Три варианта задачи о рюкзаке	4
3 Задача 3. Две корзины	6
4 Задача 4. Самая удалённая вершина в поддереве и наддереве	7
5 Задача 5. Муравей на тетраэдре	8
6 Задача 6. Слоистый граф и остаток суммы весов	9
7 Задача 7. Пути под кусочно-постоянным потолком	10
8 Задача 8. Неоднородная линейная рекуррентность	11
9 Задача 9. Гладкие числа за логарифмическое время	12
10 Задача 10. Пути ровно из k рёбер	13
11 Задача 11. ННП в периодическом массиве	14

Обозначения и соглашения

Во всех оценках n обозначает размер входа, если в условии конкретной задачи не сказано иначе. Значение $+\infty$ используется для недостижимого состояния в задачах минимизации, а $-\infty$ — в задачах максимизации. Слово «путь» может означать маршрут с повторениями вершин только там, где это прямо следует из постановки; во всех остальных случаях путь простой.

1. Задача 1. Операции с битовыми масками

Решение. Зафиксируем универсум $U = \{0, \dots, n - 1\}$ и будем считать, что используются только младшие n битов.

- а) Разность $A \setminus B$ должна оставить бит тогда и только тогда, когда он установлен в A и не установлен в B . Поэтому

$$\text{mask}(A \setminus B) = A \& (\sim B).$$

Чтобы старшие биты машинного слова не мешали, можно записать

$$A \& ((2^n - 1) \oplus B).$$

Эквивалентная формула, не использующая отрицание:

$$A \setminus B = A \oplus (A \& B).$$

- б) Ненулевая маска состоит из одного бита тогда и только тогда, когда

$$\text{mask} \neq 0 \quad \text{и} \quad (\text{mask} \& (\text{mask} - 1)) = 0.$$

Вычитание единицы меняет самый младший установленный бит на ноль, а все младшие нулевые биты — на единицы. Поэтому операция $\&$ удаляет ровно самый младший установленный бит. Результат равен нулю ровно тогда, когда других установленных битов не было.

Доказательство корректности. Первая формула проверяется независимо для каждого бита по таблице истинности $a \wedge \neg b$. Во втором пункте описанное действие $\text{mask} - 1$ показывает: при одном установленном бите пересечение пусто, а при двух или более все установленные биты, кроме младшего, остаются в пересечении. Отдельная проверка $\text{mask} \neq 0$ нужна, поскольку нулевая маска также удовлетворяет $\text{mask} \& (\text{mask} - 1) = 0$ в беззнаковой машинной арифметике. $\square$

Сложность. Все операции занимают $\mathcal{O}(1)$ времени и памяти для маски, помещающейся в одно машинное слово.

2. Задача 2. Три варианта задачи о рюкзаке

Пусть предмет i имеет вес $w_i > 0$, ценность c_i , а вместимость рюкзака равна W . Требуется максимизировать ценность при весе не больше W .

Пункт Ограниченное число копий

Решение. Пусть $dp_i[x]$ — максимальная ценность набора веса не больше x , составленного из предметов $1, \dots, i$. Тогда

$$dp_i[x] = \max_{0 \leq q \leq \min(cnt_i, \lfloor x/w_i \rfloor)} (dp_{i-1}[x - qw_i] + qc_i). \quad (1)$$

Начальное значение $dp_0[x] = 0$. Формула непосредственно даёт алгоритм

$$\mathcal{O}\left(\sum_{i=1}^n \sum_{x=0}^W \min\left(cnt_i, \left\lfloor \frac{x}{w_i} \right\rfloor\right)\right) \subseteq \mathcal{O}\left(W \sum_i cnt_i\right).$$

Часто удобна бинарная декомпозиция: представить cnt_i как сумму $1, 2, 4, \dots$ и, возможно, остатка, после чего каждую группу считать одним предметом веса qw_i и цены qc_i . Получается обычный рюкзак 0/1 за $\mathcal{O}(W \sum_i \log(cnt_i + 1))$.

Пункт Неограниченное число копий

Решение. Используем один массив $dp[x]$. Для каждого типа i просматриваем $x = w_i, w_i + 1, \dots, W$ по возрастанию:

$$dp[x] \leftarrow \max(dp[x], dp[x - w_i] + c_i). \quad (2)$$

Когда вычисляется $dp[x]$, значение $dp[x - w_i]$ уже могло использовать тот же предмет i , поэтому число его копий не ограничено. При недопустимых отрицательных ценностях можно также ничего не брать, что обеспечивается инициализацией нулями.

Пункт Дробный рюкзак

Решение. Для каждого предмета вычислим удельную ценность $\rho_i = c_i/w_i$. Сортируем предметы по невозрастанию ρ_i , после чего берём каждый целиком, пока он помещается, а от первого непомещающегося берём долю, заполняющую остаток вместимости. Предметы отрицательной удельной ценности брать не нужно, если заполнение рюкзака не обязательно.

Доказательство корректности. Формула (1) перебирает точное число копий последнего типа. После фиксации q остаток является оптимальной подзадачей на первых $i - 1$ типах, так что индукция по i доказывает ограниченный вариант. Формула (2) реализует то же разбиение по последней добавленной копии, а возрастающий порядок разрешает повторное использование типа.

Для дробного варианта применим обменный аргумент. Пусть решение использует положительную долю предмета j с $\rho_j < \rho_i$, но предмет i взят не полностью. Перенесём малый вес δ с j на i . Общий вес не изменится, а ценность возрастёт на $\delta(\rho_i - \rho_j) > 0$, что противоречит оптимальности. Следовательно, в оптимальном решении все предметы большей плотности заполняются раньше предметов меньшей плотности; жадный алгоритм имеет именно такой вид. $\square$

Сложность. Прямая ограниченная динамика имеет указанную в (1) сложность; бинарная декомпозиция — $\mathcal{O}(W \sum_i \log(cnt_i + 1))$. Неограниченный вариант: $\mathcal{O}(nW)$ времени и $\mathcal{O}(W)$ памяти. Дробный вариант: $\mathcal{O}(n \log n)$ времени и $\mathcal{O}(n)$ памяти на сортировку.

3. Задача 3. Две корзины

Решение. Белая корзина однозначно задаётся подмножеством предметов X ; чёрная получает дополнение. Пусть $S = \sum_i w_i$. Сначала обычным рюкзаком посчитаем

$$dp[x] = \#\{X \subseteq \{1, \dots, n\} : \sum_{i \in X} w_i = x\}$$

лишь для $0 \leq x < k$. Инициализация $dp[0] = 1$; для каждого предмета значения обновляются по убыванию x .

Если $S < 2k$, допустимого разложения нет: суммы двух корзин в сумме дают S , а обе не могут быть не меньше k .

Пусть $S \geq 2k$. События

$$A = \{\text{вес белой корзины} < k\}, \quad B = \{\text{вес чёрной корзины} < k\}$$

не пересекаются, поскольку их одновременное выполнение дало бы $S < 2k$. По симметрии $|A| = |B| = \sum_{x=0}^{k-1} dp[x]$. Поэтому

$$\boxed{\text{ans} = 2^n - 2 \sum_{x=0}^{k-1} dp[x]}. \quad (3)$$

Формула относится к различимым белой и чёрной корзинам, как в условии.

Доказательство корректности. Всего имеется 2^n способов выбрать белое подмножество. При $S \geq 2k$ разложение недопустимо ровно тогда, когда выполнено одно из двух непересекающихся событий A, B . Динамика по весу точно считает $|A|$. Отображение $X \mapsto \bar{X}$ является биекцией между A и B , поэтому $|B| = |A|$. Вычитание их мощностей из 2^n даёт (3) и считает каждый допустимый способ ровно один раз. $\square$

Сложность. Достаточно хранить веса $0, \dots, k-1$: $\mathcal{O}(nk)$ времени и $\mathcal{O}(k)$ памяти.

4. Задача 4. Самая удалённая вершина в поддереве и наддереве

Решение. Подвесим дерево за заданный корень. Для вершины v обозначим через $down[v]$ максимальное расстояние от v до вершины её поддерева (вершина v также разрешена). Тогда

$$down[v] = \max \left(0, \max_{u: parent[u]=v} (w(v, u) + down[u]) \right). \quad (4)$$

Вместе со значением храним вершину, где максимум достигается. Формула вычисляется обходом снизу вверх.

Теперь $up[v]$ — максимальное расстояние от v до вершины, не лежащей в строгом поддереве v ; разрешим также саму v , поэтому $up[root] = 0$. Для каждого v сохраним два наибольших значения

$$cand(u) = w(v, u) + down[u]$$

по его детям. Для ребёнка u лучший путь наружу сначала идёт по ребру (u, v) , а после v выбирает один из трёх вариантов:

1. остановиться в v , получив добавку 0;
2. продолжиться к оптимальной вершине над v , получив $up[v]$;
3. пойти в поддерево другого ребёнка $z \neq u$, получив $\max_{z \neq u} cand(z)$.

Следовательно,

$$up[u] = w(u, v) + \max \left(0, up[v], \max_{z \neq u} (w(v, z) + down[z]) \right). \quad (5)$$

Максимум без u получается за $\mathcal{O}(1)$ из двух лучших кандидатов. Второй обход идёт сверху вниз.

Доказательство корректности. Любой путь из v к вершине его поддерева либо имеет длину 0, либо первым ребром идёт к единственному ребёнку u , а остаток является нисходящим путём из u . Это доказывает (4) индукцией снизу вверх.

Рассмотрим путь из ребёнка u к вершине вне его строгого поддерева. Его первое ребро обязано быть (u, v) . После достижения v искомая вершина равна v , находится вне поддерева v , либо находится в поддереве ровно одного брата $z \neq u$. Эти случаи попарно исчерпывают возможности и дают три кандидата в (5). Каждый кандидат соответствует реальному пути, а других путей нет; индукция сверху вниз доказывает точность up . $\square$

Сложность. Каждое ребро рассматривается постоянное число раз: $\mathcal{O}(n)$ времени и $\mathcal{O}(n)$ памяти.

5. Задача 5. Муравей на тетраэдре

Решение. Граф вершин тетраэдра есть K_4 . Пусть A_t — число маршрутов длины t , заканчивающихся в исходной вершине s , а B_t — число маршрутов, заканчивающихся в одной *фиксированной* другой вершине. По симметрии это число одинаково для трёх других вершин. Тогда

$$A_{t+1} = 3B_t, \quad B_{t+1} = A_t + 2B_t,$$

и

$$\begin{pmatrix} A_{t+1} \\ B_{t+1} \end{pmatrix} = \begin{pmatrix} 0 & 3 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} A_t \\ B_t \end{pmatrix}, \quad \begin{pmatrix} A_0 \\ B_0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

Значит,

$$\begin{pmatrix} A_n \\ B_n \end{pmatrix} = \begin{pmatrix} 0 & 3 \\ 1 & 2 \end{pmatrix}^n \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

Матрица возводится в степень бинарным методом.

Доказательство корректности. Чтобы прийти в s на шаге $t + 1$, на шаге t нужно находиться в одной из трёх остальных вершин; отсюда $A_{t+1} = 3B_t$. Чтобы прийти в фиксированную вершину $v \neq s$, перед последним шагом можно находиться в s или в одной из двух прочих вершин; отсюда второе равенство. Начальное состояние учитывает единственный пустой маршрут в s . Индукция по t поэтому доказывает, что первая координата матричного произведения есть точное число требуемых маршрутов. $\square$

Сложность. Умножаются матрицы фиксированного размера 2×2 : $\mathcal{O}(\log n)$ времени и $\mathcal{O}(1)$ памяти.

6. Задача 6. Слоистый граф и остаток суммы весов

Решение. Пусть вес любого ребра, входящего в вершину j следующего слоя, равен w_j . Для остатка $q \in \mathbb{Z}/M\mathbb{Z}$ обозначим

$$c_q = \#\{j : w_j \equiv q \pmod{M}\}.$$

Состояние $v_t[r]$ равно числу способов после t переходов набрать остаток r . Переход имеет вид

$$v_{t+1}[r'] = \sum_{r=0}^{M-1} v_t[r] c_{(r'-r) \bmod M}. \quad (6)$$

Определим матрицу

$$T_{r',r} = c_{(r'-r) \bmod M}.$$

Если начальную вершину первого слоя можно выбрать произвольно, то $v_0 = ne_0$; если она фиксирована, $v_0 = e_0$. После $l-1$ рёбер

$$v_{l-1} = T^{l-1}v_0,$$

и ответ равен координате $v_{l-1}[0]$. При $v_0 = ne_0$ в ней уже просуммированы все варианты начальной и конечной вершин.

Доказательство корректности. При переходе из остатка r в r' нужно выбрать вершину следующего слоя с весом $q \equiv r' - r \pmod{M}$; таких вершин ровно c_q . Поэтому (6) учитывает все и только допустимые продолжения каждого пути. Индукция по числу межслойных переходов доказывает смысл вектора v_t . Возведение матрицы в степень лишь группирует последовательное применение того же точного перехода и не меняет результата. $\square$

Сложность. Подсчёт c_q занимает $\mathcal{O}(n)$ (и укладывается в заявленное $\mathcal{O}(nM)$). Бинарное возведение матрицы $M \times M$ в степень требует $\mathcal{O}(M^3 \log l)$ времени и $\mathcal{O}(M^2)$ памяти.

7. Задача 7. Пути под кусочно-постоянным потолком

Решение. Состоянием служит текущая высота $0, \dots, Y$. Для потолка высоты h введём матрицу T_h размера $(Y + 1) \times (Y + 1)$:

$$(T_h)_{q,p} = \begin{cases} 1, & 0 \leq p, q \leq h \text{ и } |p - q| \leq 1, \\ 0, & \text{иначе.} \end{cases} \quad (7)$$

Её столбец p описывает один шаг из высоты p в высоту q .

Пусть i -й горизонтальный отрезок задаёт потолок h_i на $[a_i, b_i]$, а $L_i = b_i - a_i$. Начинаем с вектора $v = e_0$ и последовательно выполняем

$$v \leftarrow T_{h_i}^{L_i} v, \quad i = 1, \dots, n. \quad (8)$$

Если значение потолка в общей граничной точке относится к обоим отрезкам, достаточно после каждого блока занулить координаты выше $\min(h_i, h_{i+1})$; эквивалентно можно заранее договориться о полуинтервалах $[a_i, b_i)$ и отдельно проверить конечные точки. Ответ — координата $v[0]$.

Доказательство корректности. По определению (7), ненулевой элемент матрицы соответствует ровно одному разрешённому шагу: абсцисса увеличивается на единицу, высота меняется на $-1, 0$ или 1 , остаётся неотрицательной и не превосходит текущий потолок. Элемент $(T_h^L)_{q,p}$ по правилу матричного умножения равен числу последовательностей из L таких шагов из p в q . Следовательно, (8) точно пересчитывает число путей на очередном отрезке. Индукция по отрезкам показывает, что после последнего из них $v[0]$ считает все и только пути из $(0, 0)$ в $(k, 0)$. $\square$

Сложность. Каждая из n матриц возводится в степень за $\mathcal{O}(Y^3 \log L_i) \subseteq \mathcal{O}(Y^3 \log k)$. Общее время $\mathcal{O}(nY^3 \log k)$, память $\mathcal{O}(Y^2)$.

8. Задача 8. Неоднородная линейная рекуррентность

Решение. Дано

$$a_0 = 13, \quad a_1 = 8, \quad a_n = 5a_{n-1} + 2a_{n-2} + n^2.$$

Добавим в состояние полиномиальные величины:

$$X_n = (a_n, a_{n-1}, n^2, n, 1)^T.$$

Тогда

$$X_{n+1} = MX_n, \quad M = \begin{pmatrix} 5 & 2 & 1 & 2 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 2 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}. \quad (9)$$

Первая строка использует $a_{n+1} = 5a_n + 2a_{n-1} + (n+1)^2$, а третья — $(n+1)^2 = n^2 + 2n + 1$. Для $k \geq 1$

$$X_k = M^{k-1}(8, 13, 1, 1, 1)^T.$$

При $k = 0$ непосредственно возвращаем 13.

Доказательство корректности. Каждая координата произведения MX_n по (9) равна соответствующей координате X_{n+1} : первая — по исходной рекуррентности, вторая переносит a_n , последние три реализуют тождества для $n+1$. Следовательно, индукцией $X_k = M^{k-1}X_1$. Первая координата этого вектора и есть требуемое a_k . $\square$

Сложность. Размер матрицы постоянен, поэтому бинарное возведение требует $\mathcal{O}(\log k)$ времени и $\mathcal{O}(1)$ памяти.

9. Задача 9. Гладкие числа за логарифмическое время

Решение. Построим матрицу переходов между цифрами:

$$T_{d,e} = \begin{cases} 1, & |d - e| \leq 1, \\ 0, & \text{иначе.} \end{cases}$$

Пусть столбец v_1 имеет $v_1[d] = 1$ для $d = 1, \dots, 9$ и $v_1[0] = 0$. Тогда

$$v_n = T^{n-1}v_1, \quad \text{ans} = \sum_{d=0}^9 v_n[d].$$

Для $n = 1$ используется $T^0 = I$.

Доказательство корректности. Координата $v_t[d]$ равна числу допустимых t -значных чисел с последней цифрой d . Это верно при $t = 1$. Умножение на T суммирует ровно по тем предыдущим цифрам e , для которых новая соседняя пара удовлетворяет ограничению. Индукция доказывает смысл всех слоёв, а сумма по последней цифре даёт ответ. $\square$

Сложность. Бинарное возведение матрицы 10×10 занимает $\mathcal{O}(\log n)$ времени и $\mathcal{O}(1)$ памяти в асимптотике по n .

10. Задача 10. Пути ровно из k рёбер

Решение. Работаем в полукольце *max-plus*:

$$x \oplus y = \max(x, y), \quad x \otimes y = x + y.$$

Нулём для $\oplus$ служит $-\infty$, единицей для $\otimes$ — 0. Для матриц

$$(A \odot B)_{ij} = \max_{v=1}^n (A_{iv} + B_{vj}). \quad (10)$$

Пусть W_{ij} равно максимальному весу ребра $i \rightarrow j$, а при отсутствии ребра — $-\infty$. Тогда искомая матрица равна

$$\boxed{W^{\odot k}}.$$

Нулевая степень есть max-plus единичная матрица: нули на диагонали и $-\infty$ вне её. Степень вычисляется бинарным возведением с операцией (10).

Доказательство корректности. Докажем более общее утверждение: $(W^{\odot q})_{ij}$ равно максимальному весу маршрута из i в j , содержащего ровно q рёбер. Для $q = 1$ это определение W , для $q = 0$ — определение единичной матрицы. Если матрицы A, B описывают соответственно маршруты ровно из p и q рёбер, любой маршрут из $p + q$ рёбер имеет единственную вершину v после первых p рёбер. Его оптимальный вес при фиксированном v равен $A_{iv} + B_{vj}$, а максимум по v даёт (10). Обратно, каждый конечный кандидат в (10) склеивает два существующих маршрута. Следовательно, max-plus умножение точно соответствует конкатенации, и утверждение следует индукцией. $\square$

Сложность. Одно умножение стоит $\mathcal{O}(n^3)$, бинарное возведение выполняет $\mathcal{O}(\log k)$ умножений. Итого $\mathcal{O}(n^3 \log k)$ времени и $\mathcal{O}(n^2)$ памяти.

11. Задача 11. ННП в периодическом массиве

Условие. Блок $a_1, \dots, a_n$ повторён t раз. Требуется найти длину наибольшей неубывающей подпоследовательности (ННП).

Решение. Сожмём различные значения блока в ранги $1, \dots, d$, сохранив порядок: $a_i \leq a_j$ тогда и только тогда, когда $r_i \leq r_j$. Добавим состояние 0, соответствующее пустой подпоследовательности с последним значением $-\infty$.

Перед обработкой очередной позиции храним вектор F . Полагаем $F_0 = 0$, а для $q \geq 1$ величина F_q равна максимальной длине уже выбранной подпоследовательности, заканчивающейся значением ранга q . Недостижимое состояние равно $-\infty$. При чтении элемента ранга r меняется только координата r :

$$F'_r = 1 + \max_{0 \leq q \leq r} F_q, \quad F'_q = F_q \quad (q \neq r). \quad (11)$$

В старое значение F_r уже входит в максимум, и добавление текущего элемента действительно улучшает его на единицу.

Преобразование (11) max-plus линейно: существует элементарная матрица E_r , для которой $F' = E_r \odot F$. На диагонали E_r стоят нули, а в строке r стоят единицы в столбцах $0, \dots, r$ и $-\infty$ в остальных; строка r заменяет свою диагональную нулевую ячейку на 1.

Преобразование одного полного блока равно

$$T = E_{r_n} \odot \dots \odot E_{r_1}.$$

Его можно построить за $\mathcal{O}(n^3)$, не перемножая n плотных матриц: применяем один блок к каждому из $d + 1$ max-plus базисных столбцов; одно применение (11) требует $\mathcal{O}(d)$ времени на максимум, то есть всего $\mathcal{O}(n(d + 1)^2) = \mathcal{O}(n^3)$.

Для начального вектора $F^{(0)} = (0, -\infty, \dots, -\infty)^T$ после t блоков

$$F^{(t)} = T^{\odot t} \odot F^{(0)}.$$

Ответ равен $\max_q F_q^{(t)}$.

Доказательство корректности. Докажем инвариант для (11). Любая ННП, заканчивающаяся текущим значением ранга r , получается добавлением текущего элемента к ННП, последний ранг которой не превосходит r ; лучший такой префикс имеет длину $\max_{q \leq r} F_q$. Обратно, к каждому такому префиксу текущий элемент можно добавить, поэтому правая часть достижима. Подпоследовательности с другим последним рангом текущий элемент не используют, и их оптимумы не меняются. Значит, (11) пересчитывает состояния точно.

Матрица T есть последовательное применение точных переходов ко всем позициям одного блока. Ассоциативность max-plus умножения означает, что $T^{\odot t}$ применяет тот же блок t раз, то есть обрабатывает ровно исходный массив длины nt . Наконец, любая непустая ННП заканчивается некоторым рангом, поэтому максимум координат равен её оптимальной длине; координата 0 корректно покрывает пустой случай. $\square$

Сложность. Сжатие значений занимает $\mathcal{O}(n \log n)$, построение T — $\mathcal{O}(n^3)$, max-plus возведение — $\mathcal{O}(n^3 \log t)$. Итоговое время $\mathcal{O}(n^3 \log t)$, память $\mathcal{O}(n^2)$.